REPORT

Task 1: Dynamic Array Simulation (Like Python List)

2. Analysis:

• Explain amortized time complexity of append() in simple words.

• Mention: most appends are O(1), but resizing makes some appends O(n); overall amortized O(1).

= In simple terms, **amortized time complexity** is a way to describe the average time an operation takes over a long sequence of events, even if one specific instance of that operation is much slower than the others.

When you use append() to add an element to a dynamic array (like a Python list):

- **The common case O(1):** Most of the time, there is extra space already allocated in the underlying memory. Adding a new item is "constant time" or **O(1)** because the computer just places the value in the next available slot.
- **The rare case O(n):** Eventually, the array runs out of pre-allocated space. When this happens, the computer must "resize" the array by creating a new, larger space in memory and copying all n existing elements into it. This specific append is slow and takes **O(n)** time.

Conclusion

Even though the resizing step is expensive, it happens very infrequently (usually only when the array doubles in size). Because the cost of that one slow O(n) operation is spread out, or "amortized," across all the fast O(1) operations that came before it, the **overall amortized time complexity is O(1)**.

Task 3: Stack & Queue From Scratch (Using Singly Linked List)

(C) Complexity Discussion (write in report.pdf)

Write time complexity of each operation in simple words:

• Stack: push/pop/peek

= Stack Operations

A stack follows the **LIFO** (Last-In, First-Out) principle. All operations happen at the "top" of the stack, which means we never have to shift elements or search through the data structure.

- **Push : O(1)** – Adding an item to the top is a direct operation.
- **Pop :O(1)** – Removing the top item is a direct operation.
- **Peek : O(1)** – Looking at the top item without removing it takes constant time.

• Queue: enqueue/dequeue/front

= A queue follows the **FIFO** (First-In, First-Out) principle. In an efficient implementation (like a linked list or a circular buffer), we keep track of both the "front" and the "rear."

- **enqueue: $O(1)$** – Adding an item to the back of the line is a direct operation.
- **dequeue: $O(1)$** – Removing the item from the front of the line is a direct operation.
- **front: $O(1)$** – Accessing the first element in line takes constant time.\

Operation	Stack (LIFO)	Queue (FIFO)
Insertion	$O(1)$ (push)	$O(1)$ (enqueue)
Removal	$O(1)$ (pop)	$O(1)$ (dequeue)
Access/View	$O(1)$ (peek)	$O(1)$ (front)

Summary:

Amortized Analysis of append()

In a dynamic array, most append operations are **O(1)** because space is already available. However, when the capacity is reached, the array must be resized (doubled). This specific operation is **O(n)** because every existing element must be copied to the new memory block.

- **Why Amortized O(1)?** The costly O(n) resize happens very infrequently (only after n appends). When you spread the cost of one resize over all the preceding O(1) operations, the "average" cost per operation remains constant

Data Structure	Operation	Complexity
Stack	Push / Pop	$O(1)$
Queue	Enqueue	$O(1)$
Queue	Dequeue	$O(1)$
Linked List	Search	$O(n)$